

Title:

Intelligent Document Extraction, Validation & API Platform

AI Engineer Internship – Technical Case Study

Candidate: Ananthi R
Document Intelligence Solution

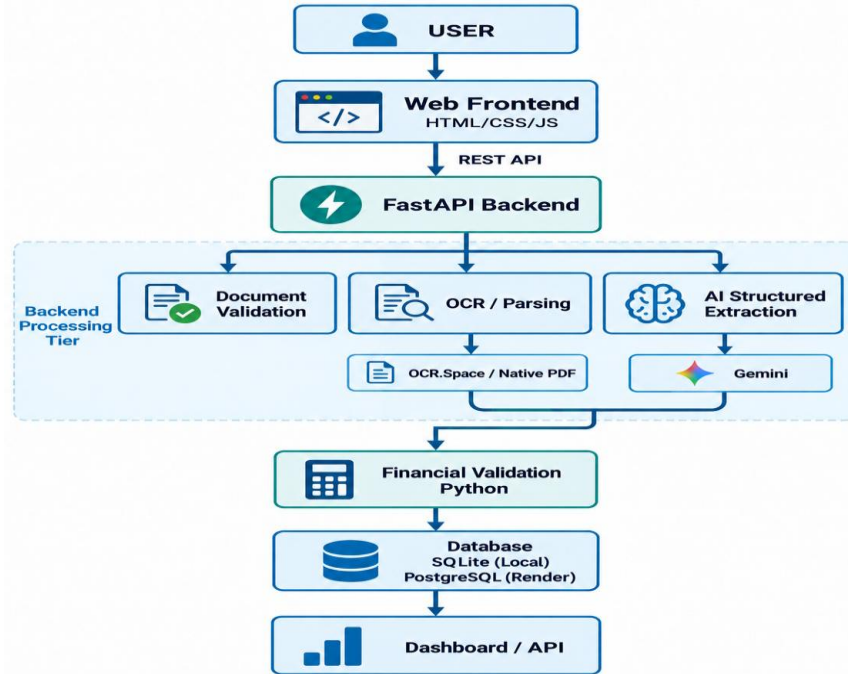
Python / FastAPI / OCR / Gemini / PostgreSQL / SQLite

Problem Statement & Objective

- Financial documents arrive in different layouts and formats.
- Documents may be native PDFs or scanned PDF/JPG/PNG images.
- OCR errors and varying financial tables make manual extraction difficult.
- The solution must extract all meaningful document information into structured JSON.
- Financial relationships must be validated using deterministic calculations.
- Processed results must be stored and accessible through REST APIs and a dashboard.



Solution Architecture



- Modular architecture with separate validation, OCR, extraction, validation, repository and API layers.
- PostgreSQL stores deployed results on Render. SQLite is used locally
- AI is used for extraction; deterministic Python is used for financial calculations.

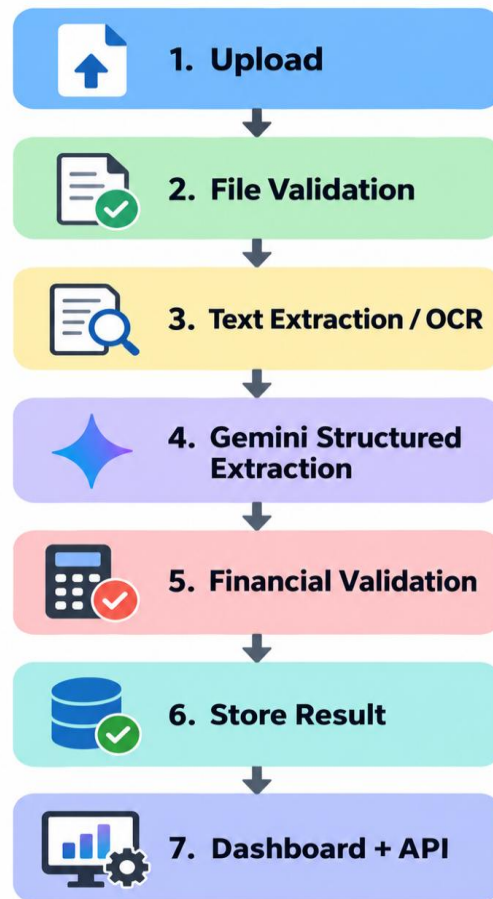
Processing Pipeline

File Validation

- PDF / JPG / PNG only
- Maximum 3 pages
- Empty/corrupted file handling
- Basic file integrity check

OCR Strategy

- Native PDF text extraction first
- OCR.Space for scanned/image documents
- Gemini Vision fallback when required



Supported Documents & Structured Extraction

Invoice

- Invoice details
- Vendor / customer
- Currency and totals
- Line items
- Discount / tax / rounding
- Evidence

Balance Sheet

- Statement headers
- Reporting periods
- Assets
- Liabilities
- Equity
- Complete line items

Profit & Loss

- Income
- Expenses
- Profit values
- Comparative periods
- Appropriation information

Cash Flow

- Operating activities
- Investing activities
- Financing activities
- Opening / closing cash
- FX / adjustments

Missing or unsupported values → **null**

No guessing or invented values.

Financial Validation

Financial calculations are performed using Python, not the LLM.

Invoice

$\text{Quantity} \times \text{Unit Price} \approx \text{Line Amount}$

$\text{Sum}(\text{Line Amounts}) \approx \text{Subtotal}$

$\text{Taxable Amount} + \text{Tax} + \text{Round Off} \approx \text{Total}$

$\text{Cash Paid} - \text{Total} \approx \text{Change}$

Balance Sheet

$\text{Capital \& Liabilities} \approx \text{Total Assets}$

Profit & Loss

$\text{Interest Earned} + \text{Other Income} \approx \text{Total Income}$

$\text{Total Income} - \text{Total Expenditure}$

$\approx \text{Net Profit before Minority Interest}$

Cash Flow

$\text{OCF} + \text{ICF} + \text{FCF} + \text{FX}$

$\approx \text{Net Change in Cash}$

$\text{Opening Cash} + \text{Net Change} + \text{Adjustments} \approx \text{Closing Cash}$

Financial Validation – PASS & FAIL Examples

Successful validation -PASS Case – Invoice

20251118_000612.jpg

Validation Check	Result
Line-item calculations	PASS
Discount handling	PASS
Line items → Subtotal	PASS
Taxable Amount + Tax + Round Off	PASS
Invoice Total	PASS

Taxable Amount = 5,815.17

Tax Amount = 1,046.72

Round Off = 0.11

$5,815.17 + 1,046.72 + 0.11$

= 6,862.00

Result: PASS 

Real failure detected

FAIL Case – Invoice batch2-0499.jpg

Sum of line-item amounts = 145.00

Reported subtotal = 135.00

Variance = 10.00

Result = FAIL 

The system does not force documents to PASS.

It reports genuine financial inconsistencies found in the source document.

APIs, Dashboard & Persistence

API'S

POST /api/v1/documents/process

GET /api/v1/documents/{document_name}

GET /api/v1/documents

GET /api/v1/health

Dashboard

- Document type selection
- PDF/JPG/PNG upload
- Processing status
- Extracted key-value fields
- Line-item tables
- Financial validation results
- Raw JSON viewer
- Database-backed processed-document list

FastAPI automatically provides Swagger/OpenAPI documentation at [/docs](#).

Technology Stack & Engineering Decisions

Component	Technology
Backend	FastAPI / Python
Structured schemas	Pydantic
AI extraction	Gemini
OCR	OCR.Space + Gemini Vision fallback
PDF handling	PyPDF / PyMuPDF
Database	PostgreSQL + SQLite + SQLAlchemy
Frontend	HTML / CSS / JavaScript
API documentation	Swagger / OpenAPI
Version control	GitHub
Deployment	Render

Key Design Decisions

- User selects document type — no unnecessary auto-classifier.
- LLM handles unstructured extraction.
- Python handles deterministic arithmetic.
- Evidence/source text improves traceability.
- Modular services improve maintainability.
- Environment variables protect API keys.

Document Intelligence

AI-powered financial document extraction and validation

Process Document

Upload a PDF, JPG or PNG document and select its document type.

Document Type

Invoice



Document

Choose File

batch2-0469.jpg

Process Document

Document processed successfully.

Additional Fields

Name	Value
vendor_address	101 B Devon Road BRAMPTON
vendor_email	KiranP@taylor-corp.ca
customer_address	160 Markland Street MARKHAM, Ontario, Canada L6C 0C6
customer_email	ap@ohi.net
terms_and_conditions	Please send payment within 30 days of receiving this invoice. There will be a 1.5% interest charge per month on late invoices.

Financial Validation

Total: 7

Passed: 6

Failed: 1

N/A: 0

Check	Formula	Calculated	Reported	Variance	Status
Line item 1 calculation	quantity × unit_price = amount	16	16	0	PASS
Line item 2 calculation	quantity × unit_price = amount	56	56	0	PASS
Line item 3 calculation	quantity × unit_price = amount	30	30	0	PASS
Line item 4 calculation	quantity × unit_price = amount	28	28	0	PASS
Line item 5 calculation	quantity × unit_price = amount	15	15	0	PASS
Line items to subtotal	sum(line_item.amount) = subtotal	145	135	10	FAIL
Invoice total reconciliation	subtotal + tax_amount + shipping_handling - discount + round_off = total	157.48	157.48	0	PASS

Testing, Limitations & Deployment

Testing

- Tested all four document types.
- Tested scanned/image invoices.
- Tested valid financial reconciliation.
- Tested real financial validation failure.
- Tested missing values.
- Tested OCR fallback.
- Invalid/unsupported file test.
- Basic automated tests

Current Limitations

- OCR accuracy depends on source image quality.
- Free-tier API/model quotas may affect availability.
- Some complex table layouts can produce row alignment issues.
- SQLite is used locally; deployed persistence uses PostgreSQL on Render.
- Confidence scoring not implemented.

Production Improvements

- PostgreSQL / managed database
- Background task queue
- More robust table extraction
- Authentication / authorization
- Monitoring and metrics
- Automated evaluation dataset
- Containerized deployment

GitHub:

<https://github.com/Ananthi0207/document-intelligence>

Frontend:

<https://document-intelligence-fchk.onrender.com/>

API:

<https://document-intelligence-fchk.onrender.com>

Swagger:

<https://document-intelligence-fchk.onrender.com/docs>

Conclusion

- Built an end-to-end document intelligence prototype supporting four financial document types.
- Combined OCR + LLM extraction with deterministic financial validation and evidence-grounded structured output.
- Delivered results through REST APIs, persistent storage and a web dashboard.
- Implemented source-grounded extraction with evidence text and page references to improve traceability.
- Added database-backed persistence so processed documents can be retrieved later and viewed through the dashboard.
- Designed the solution in a modular way so OCR, AI extraction, financial validation, API, and database layers can be improved independently.